



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Pablo Arturo Gonzalez Cuellar

N° de Cuenta: 319241013

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

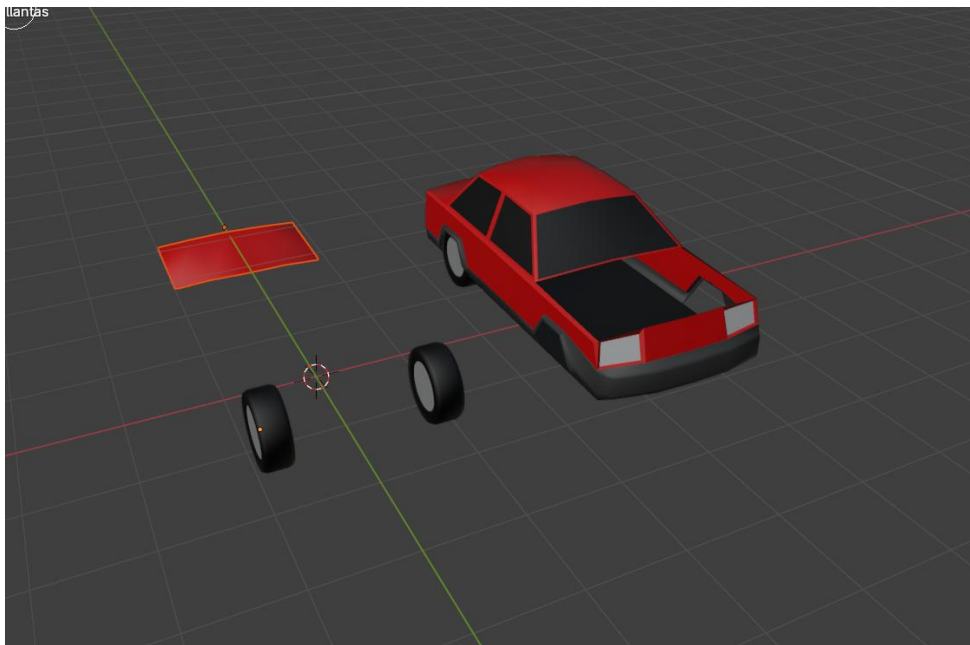
FECHA DE ENTREGA LÍMITE: 28/09/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

- 1.- Importar su modelo de coche propio dentro del escenario a una escala adecuada.
- 2.- Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.
- 3.- Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.
- 4.- Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente

Para la realización de esta práctica, el objetivo principal fue cargar un modelo propio de un carro en 3D, el cuál mediante la herramienta Blender, se descompuso en 3 de sus componentes principales, el chasis, las llantas y el cofre, esto para posteriormente exportarlo a OpenGL y realizar los ejercicios de la práctica.



Como primer paso, una vez descompuesto el modelo, se exportó en formato fbx y se cargó a OpenGL dentro de nuestro archivo main donde precisamente se crearon tres objetos de la clase *Model* donde cada uno cargó su respectivo archivo.

```
// Declaramos un objeto Model para cada parte del coche
Model carroceria;
Model llantas;
Model cofre;
```

Una vez cargados los modelos, declaramos las variables necesarias para la traslación del coche completo, para la rotación de las llantas y una última para la rotación del cofre

```
// Getters para el coche
GLfloat getMovCoche() { return movCoche; }
GLfloat getRotLlantas() { return rotLlantas; }
GLfloat getRotCofre() { return rotCofre; }

~Window();

private:
    GLFWwindow* mainWindow;
    GLint width, height;

// Variables para el coche
GLfloat movCoche;
GLfloat rotLlantas;
GLfloat rotCofre;
```

Ahora bien, una vez declaradas estas variables y funciones, se implementó la lógica de control de todo el escenario, donde definimos que las teclas “E” y “R” incrementan “*movCoche*” la cual más adelante veremos que será la encargada de mover hacia delante o hacia atrás el vehículo. Las teclas “1” y “2” las asignamos para la rotación de las llantas y por ultimo las teclas “3” y “4” para la rotación del cofre. Además se agregó un límite de rotación del cofre para que este no tuviera un movimiento extraño hacia abajo.

```
if (action == GLFW_PRESS || action == GLFW_REPEAT)
{
    // Movimiento de todo el coche
    if (key == GLFW_KEY_E) {
        theWindow->movCoche += 0.1f;
    }
    if (key == GLFW_KEY_R) {
        theWindow->movCoche -= 0.1f;
    }

    // Rotación de las llantas
    if (key == GLFW_KEY_1) {
        theWindow->rotLlantas += 2.0f;
    }
    if (key == GLFW_KEY_2) {
        theWindow->rotLlantas -= 2.0f;
    }

    // Cofre
    if (key == GLFW_KEY_3)
    {
        // límite de 45 grados para abrir
        if (theWindow->rotCofre < 45.0f) {
            theWindow->rotCofre += 2.0f;
        }
    }
}
```

Una vez implementado el sistema de control, el siguiente paso fue utilizar todas estas variables en nuestro main para darle vida a los modelos. Primero se creó una matriz auxiliar la cual se encargará de guardar la posición, escala y orientación de todo el coche para facilitar el movimiento total del coche posteriormente

```
// Matriz maestra para orientar y mover todo el coche
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, mainWindow.getMovCoche())); /
model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
modelaux = model;

// Carrocería
model = modelaux;
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.5f, 0.0f, 0.8f)));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
carroceria.RenderModel();
```

Así pues, se fue construyendo pieza por pieza el coche, partiendo de la matriz auxiliar asegurando tener una implementación jerárquica que aunque cada pieza se exportó individualmente y tiene su propio eje, todas permanecen ancladas a esta matriz y se mueven con ella.

```
// Cofre
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 1.05f, 0.65f));
model = glm::rotate(model, glm::radians(-mainWindow.getRotCofre()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.8f, 0.1f)));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cofre.RenderModel();
```

```
// LLANTAS
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(0.0f, 0.0f, 0.0f)));

// Llantas traseras
model = modelaux;
model = glm::translate(model, glm::vec3(-0.9f, 0.3f, -1.25f));
model = glm::rotate(model, glm::radians(mainWindow.getRotLlantas()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantas.RenderModel();

// Llantas delanteras
model = modelaux;
model = glm::translate(model, glm::vec3(-0.9f, 0.3f, 1.3f));
model = glm::rotate(model, glm::radians(mainWindow.getRotLlantas()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantas.RenderModel();
```

CAPTURAS DE EJECUCIÓN:



PROBLEMAS SURGIDOS:

Afortunadamente, en esta práctica no se tuvo ningún problema, solamente algunos problemas al exportar los objetos debido a que no recordaba la configuración al exportar modelos en Blender y por ende al importar los modelos resultaba que estaba exportando el mismo modelo pero con diferente nombre de archivo.

CONCLUSIÓN

Se reforzó la herramienta tan importante que son los software para el modelado de nuestros objetos, así como también la importancia de una buena planificación al momento de exportar pieza por pieza estos modelados ya que son fundamentales para poder manipularlas de forma independiente en el código.

Además, práctica con práctica se vuelve algo fundamental aplicar un sistema de transformación jerárquico, en el cual definimos una matriz auxiliar para guardar información clave para así tener un resultado favorecedor. En esta práctica, por ejemplo, permitió que, al mover el coche completo, todas sus partes se movieran con él, pero que a su vez, cada componente pudiera tener su propia “animación” como la rotación del cofre.